Optimization Dynamics Laboratory

Technical Report

Nicolas Iskandar

April 29, 2026

Abstract

The Optimization Dynamics Laboratory is a Python framework for the empirical study of numerical optimization. It implements gradient-based, line-search, second-order, and constrained solvers from first principles, together with finite-difference derivatives and trajectory diagnostics. This report describes the system architecture, the numerical approximations used by the solvers, and the visualization layer used to inspect optimization dynamics.

Contents

1 System Architecture **2**

1.1 Core Modules 2

1.2 Execution Engine: TrajectoryRunner 2

2 Numerical Differentiation **4**

2.1 Gradient Approximation 4

2.2 Hessian Approximation via Gradient Differencing 4

3 Optimization Algorithms **5**

3.1 Gradient Descent 5

3.2 Momentum Gradient Descent 5

3.3 Newton's Method 6

3.4 Mini-Batch Stochastic Gradient Descent with Gaussian Noise 7

4 Line Search Methods **9**

4.1 Why Line Search is Needed 9

4.2 Global Convergence: Backtracking Line Search 9

4.3 Derivative-Free Line Search: Golden Section Search 10

5 Constrained Optimization **12**

5.1 Penalty Method 12

5.2 Lagrange Multipliers 12

6 Visualization Architecture **14**

6.1 Vectorized Evaluation 14

6.2 Divergence Handling 14

7 Experimental Observations **15**

8 Conclusion **16**

1 System Architecture

The laboratory is a modular system separating objective functions, optimizers, diagnostics, and visualization components. The design is intended to keep the numerical core transparent and to make algorithmic behavior easy to inspect.

1.1 Core Modules

- **`core/`**: Finite-difference differentiation engine for gradients and Hessians.
- **`functions/`**: Collection of 2D objective functions.
- **`optimizers/`**: First-order, second-order, and line-search methods.
- **`dynamics/`**: Trajectory storage and diagnostic computation.
- **`visualization/`**: 2D and 3D plotting utilities.

1.2 Execution Engine: TrajectoryRunner

The TrajectoryRunner (TR) class manages execution of optimization runs and stores the resulting optimization trajectory together with diagnostics.

```
objective = TrajectoryRunner._build_optimizer_objective(optimizer, function)
trajectory_array = optimizer.optimize(objective, x0, steps=steps, tol=tol)
diagnostics_data = Diagnostics.compute_trajectory_diagnostics(function.f, trajectory_array)
trajectory = Trajectory(function, optimizer_name, x0, trajectory_array, diagnostics_data)
```

Most optimizers consume a scalar callable $f(x)$. The current Stochastic Gradient Descent (SGD) implementation instead operates on a finite-sum objective represented as a list of component functions. To keep the visualization and comparison pipeline uniform, TR adapts analytical demo objectives into repeated component lists when an SGD instance is used. This preserves a single execution interface for plotting, diagnostics, and experiment modules.

Batch utilities include:

- `run_comparison`: compare multiple optimizers on the same objective.
- `run_multistart`: evaluate sensitivity to initialization.

2 Numerical Differentiation

The system avoids automatic differentiation and instead uses finite differences. This keeps the implementation compact and makes the numerical approximations explicit.

2.1 Gradient Approximation

The gradient $\nabla f(x)$ is approximated using central differences:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}, \tag{2.1}$$

where e_i is the i-th unit vector and ϵ is a small finite-difference step. This is a second-order accurate approximation in ϵ.

2.2 Hessian Approximation via Gradient Differencing

The Hessian matrix is computed as the finite difference of the gradient, i.e. as the derivative of $\nabla f(x)$. For each column i,

$$H_{:,i} \approx \frac{\nabla f(x + \epsilon e_i) - \nabla f(x - \epsilon e_i)}{2\epsilon}. \tag{2.2}$$

This is equivalent to approximating second derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}, \tag{2.3}$$

but the implementation reuses the gradient routine directly. This makes the code simpler and keeps the Hessian logic consistent with the gradient module.

3 Optimization Algorithms

3.1 Gradient Descent

Gradient descent is a first-order optimization method that updates parameters in the direction of steepest descent of the objective function.

Given an objective $f(x)$, the update rule is

$$x_{k+1} = x_k - \eta \nabla f(x_k), \tag{3.1}$$

where $\eta > 0$ is a fixed step size.

The implementation uses a finite-difference gradient approximation and stops early if the gradient norm satisfies

$$\|\nabla f(x_k)\| < \text{tol}. \tag{3.2}$$

```
grad = Gradient.get_grad(f, x)
x = x - step_size * grad
```

Remarks

- Uses a fixed learning rate throughout optimization.
- Convergence speed depends strongly on step size choice.
- Early stopping is based on gradient norm threshold.

3.2 Momentum Gradient Descent

Momentum gradient descent extends standard gradient descent by introducing a velocity term that accumulates past gradients, improving convergence on ill-conditioned problems.

The velocity update is given by

$$v_{k+1} = \beta v_k + \nabla f(x_k), \tag{3.3}$$

and the parameter update is

$$x_{k+1} = x_k - \eta v_{k+1}, \tag{3.4}$$

where $\eta > 0$ is the step size and $\beta \in [0, 1)$ controls the decay of past gradients.

The implementation stops early when

$$\|\nabla f(x_k)\| < \text{tol}. \tag{3.5}$$

```
v = beta * v + grad
x = x - step_size * v
```

Remarks

- The velocity term acts as an exponential moving average of gradients.
- Larger β increases inertia and can accelerate convergence in shallow valleys.
- Too large β may introduce oscillations or overshooting.

3.3 Newton's Method

Newton's method is a second-order optimization algorithm that uses both gradient and Hessian information to construct a local quadratic approximation of the objective function.

The update rule is

$$x_{k+1} = x_k - H(x_k)^{-1} \nabla f(x_k), \tag{3.6}$$

where $H(x_k)$ is the Hessian matrix of second derivatives.

In implementation, the Newton step is computed by solving the linear system

$$H(x_k) p_k = \nabla f(x_k), \tag{3.7}$$

and updating

$$x_{k+1} = x_k - p_k. \tag{3.8}$$

The algorithm terminates early if

$$\|\nabla f(x_k)\| < \text{tol}, \tag{3.9}$$

or if the Hessian is singular or ill-conditioned.

```
H = Hessian.get_hessian(f, x)
step = np.linalg.solve(H, grad)
x = x - step
```

Remarks

- Uses second-order curvature information for faster local convergence.
- The linear system is solved directly for numerical stability.
- May fail or break down when the Hessian is singular or indefinite.

3.4 Mini-Batch Stochastic Gradient Descent with Gaussian Noise

The current implementation follows mini-batch stochastic gradient descent with additive Gaussian noise on the gradient estimate. The method is defined on a finite-sum objective of the form

$$F(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x), \tag{3.10}$$

where each f_i is a component function.

At each iteration k, a mini-batch $B_k \subset \{1, \ldots, N\}$ of size b is sampled uniformly without replacement. The corresponding mini-batch gradient estimator is

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(x_k). \tag{3.11}$$

In addition to the stochastic gradient estimate, isotropic Gaussian noise is added:

$$\epsilon_k \sim \mathcal{N}(0, \sigma^2 I), \tag{3.12}$$

where $\sigma =$ `noise_std` controls the noise magnitude.

The parameter update rule is then

$$x_{k+1} = x_k - \eta(g_k + \epsilon_k), \tag{3.13}$$

where $\eta > 0$ is the step size.

```
batch_indices = rng.choice(n_functions, size=batch_size,
    replace=False)

batch_grads = []
for idx in batch_indices:
    fi = functions[idx]
    grad = Gradient.get_grad(fi, x)
    batch_grads.append(grad)

batch_grad = np.mean(batch_grads, axis=0)

noise = rng.normal(loc=0.0, scale=noise_std, size=x.shape)

x = x - step_size * (batch_grad + noise)
```

Remarks

- When $|B_k| = N$, the method reduces to full-batch gradient descent with additive Gaussian perturbations.
- Stochasticity arises from both minibatch sampling and injected Gaussian noise.
- The noise term is isotropic and independent across iterations.
- The stopping criterion is based on the norm of the mini-batch gradient:

$$\| g_k \| < \text{tol}, \tag{3.14}$$

  which does not include the noise term and may be sensitive to sampling variability.

4 Line Search Methods

Line search selects a step size α once a search direction d has been chosen. The full multidimensional objective is reduced to a one-dimensional function along the line

$$\phi(\alpha) = f(x + \alpha d), \tag{4.1}$$

where x is the current iterate. All line-search routines in the project operate on $\phi(\alpha)$.

4.1 Why Line Search is Needed

A fixed step size can be too large on steep or curved objectives and too small on flat ones. Line search adapts the step to the local geometry of the objective. This makes the optimizer more stable and usually more efficient than using a fixed learning rate alone.

4.2 Global Convergence: Backtracking Line Search

Backtracking line search implements the Armijo sufficient decrease condition. Starting from an initial step size α_0, the algorithm repeatedly shrinks the step by a factor $\rho \in (0, 1)$ until

$$f(x + \alpha d) \leq f(x) + c\alpha \nabla f(x)^T d \tag{4.2}$$

is satisfied, where $c \in (0, 1)$.

The quantity

$$\nabla f(x)^T d \tag{4.3}$$

is the Directional Derivative. It measures the instantaneous rate of change of the objective if one moves in direction d. If d is a descent direction, then $\nabla f(x)^T d < 0$, so the right-hand side represents a predicted decrease.

The implementation follows this logic directly:

```
f_x = f(x)
grad = Gradient.get_grad(f, x)
directional_deriv = np.dot(grad, direction)

step_size = initial_step
```

```
for _ in range(max_iters):
    f_new = f(x + step_size * direction)
    if f_new <= f_x + c * step_size * directional_deriv:
        return step_size
    step_size *= rho
```

This does not attempt to find the exact minimizer along the line. It only requires a step that decreases the objective sufficiently relative to the local linear approximation. That is enough to prevent overly aggressive updates on difficult landscapes.

4.3 Derivative-Free Line Search: Golden Section Search

Golden section search solves the one-dimensional problem

$$\min_{\alpha \in [a,b]} \phi(\alpha) = f(x + \alpha d) \tag{4.4}$$

without using gradient information. It assumes that the line-restricted objective is Unimodal Function on the interval $[a, b]$.

The method uses the constant

$$r = \frac{3 - \sqrt{5}}{2} \approx 0.381966. \tag{4.5}$$

This choice allows one previously computed function value to be reused after each interval reduction. As a result, only one new function evaluation is needed per iteration.

The interior points are initialized as

$$x_1 = a + r(b - a), \qquad x_2 = b - r(b - a), \tag{4.6}$$

with corresponding evaluations

$$f_1 = f(x + x_1 d), \qquad f_2 = f(x + x_2 d). \tag{4.7}$$

The logic is:

- If $f_1 < f_2$, the minimum is kept in the interval $[a, x_2]$.
- Otherwise, it is kept in the interval $[x_1, b]$.

After the interval is reduced, one of the old test points is reused and only one new point is computed. This is the efficiency advantage of the method.

The implementation follows the standard pattern:

```
a = 0.0
b = initial_step
x1 = a + golden_ratio * (b - a)
x2 = b - golden_ratio * (b - a)

f1 = f(x + x1 * direction)
f2 = f(x + x2 * direction)

for _ in range(max_iters):
    if abs(b - a) < tol:
        break

    if f1 < f2:
        b = x2
        x2 = x1
        f2 = f1
        x1 = a + golden_ratio * (b - a)
        f1 = f(x + x1 * direction)
    else:
        a = x1
        x1 = x2
        f1 = f2
        x2 = b - golden_ratio * (b - a)
        f2 = f(x + x2 * direction)
```

This method is useful when derivatives are unavailable or expensive, but the objective behaves reasonably well along the search direction.

5 Constrained Optimization

The system supports equality constraints $g(x) = 0$ using two distinct numerical strategies.

5.1 Penalty Method

The constrained problem is converted into an unconstrained one by adding a quadratic penalty:

$$P(x; \rho) = f(x) + \rho(g(x))^2, \tag{5.1}$$

where $\rho > 0$ controls the severity of the penalty.

Its gradient is

$$\nabla P(x) = \nabla f(x) + 2\rho g(x) \nabla g(x). \tag{5.2}$$

In the implementation, ρ is increased when the constraint violation remains too large. This forces the optimizer toward the feasible region where $g(x) \approx 0$.

5.2 Lagrange Multipliers

The standard Lagrangian is

$$L(x, \lambda) = f(x) + \lambda g(x). \tag{5.3}$$

Its gradient with respect to x is

$$\nabla_x L(x, \lambda) = \nabla f(x) + \lambda \nabla g(x). \tag{5.4}$$

The method uses a primal-dual update:

```
x = x - alpha * (grad_f + lambda * grad_g)
lambda = lambda + alpha * g(x)
```

The primal update minimizes the objective while accounting for the constraint force induced by $\lambda \nabla g(x)$. The dual update increases the multiplier when the constraint is violated and decreases it when the iterate crosses the constraint surface.

In addition to the primal-dual steps, the implementation includes a feasibility correction step. When the violation is large, the iterate is pushed directly toward the constraint manifold:

$$x \leftarrow x - \eta\, g(x) \frac{\nabla g(x)}{\|\nabla g(x)\|}. \tag{5.5}$$

This correction moves the iterate along the normal direction of the constraint surface and improves robustness when the initialization is far from feasibility.

6 Visualization Architecture

The visualization layer is intentionally non-authoritative: it renders trajectories and diagnostics produced by the execution engine rather than recomputing optimizer logic itself. This separation matters for SGD, where the finite-sum objective handling is resolved inside TR before any contour, vector-field, loss, or surface plot is created.

6.1 Vectorized Evaluation

Rendering 3D surfaces and 2D contours requires evaluating the objective on a large mesh. The implementation attempts a vectorized call first and falls back to a pointwise loop if necessary:

```
def evaluate_function(f, X, Y):
    try:
        return f(np.stack([X, Y], axis=0))
    except:
        Z = np.zeros_like(X)
        for i in range(X.shape[0]):
            for j in range(X.shape[1]):
                Z[i,j] = f(np.array([X[i,j], Y[i,j]]))
        return Z
```

6.2 Divergence Handling

Optimization runs can diverge on non-convex or poorly scaled landscapes. To prevent the visualization from being dominated by outliers, the system uses:

- Z-axis clipping to the bounds of the objective mesh.
- Removal of NaN and Inf trajectory points.

7 Experimental Observations

- Conditioning strongly affects gradient descent convergence speed.
- High momentum causes oscillatory overshoot.
- Newton's method can diverge when the Hessian is close to singular.
- Mini-batch SGD behavior depends on the component-function sampling process and becomes deterministic when the full batch is used.
- Multistart experiments reveal strong initialization sensitivity on non-convex landscapes.

8 Conclusion

The Optimization Dynamics Laboratory provides a modular environment for studying optimization dynamics through direct numerical simulation. Its value lies in the combination of transparent algorithms, explicit numerical approximations, and diagnostic visualizations that make optimization behavior easy to inspect.

Glossary

Directional Derivative Rate of change of a function along a direction, computed as the dot product of the gradient and a direction vector. 9

SGD Stochastic Gradient Descent. 2, 14, 15

TR TrajectoryRunner. 2, 14

Unimodal Function A function with a single minimum on a given interval, assumed in golden section search. 10